TECHNICAL PAPER

Design and Simulation of a Modular Siemens S7-1200 Control System for an Automated Palletizing Cell

A fault-tolerant, object-oriented PLC architecture validated through closed-loop simulation with Factory I/O and TIA Portal V16.

ElAllem Benabdelkader Ilyes

Electromechanical Engineer — Automation, Instrumentation & Industrial Control

Keywords: Siemens S7-1200 · TIA Portal · Structured Control Language (SCL) · State-Machine Programming · WinCC HMI/SCADA · Factory I/O Digital Twin · Fault-Tolerant Automation

Abstract

Summary

This paper presents the control architecture of a fault-tolerant automated palletizing cell built around a Siemens S7-1200 PLC and TIA Portal V16. The control logic was validated against a high-fidelity electromechanical plant simulation in Factory I/O, replacing open-loop timer sequences with closed-loop sensor verification and modular, object-oriented state-machine programming in Structured Control Language (SCL). Parallel diagnostic watchdogs continuously monitor actuator health to intercept mechanical failures — such as material jams — before they escalate into system faults. A centralized data structure maps directly to a WinCC HMI, providing discrete alarm management and secured manual-override capability.

1. Introduction

Industrial material-handling systems are highly susceptible to electromechanical variables such as friction, mechanical coasting, and product jams. When a control architecture relies purely on sequential timers, these physical inconsistencies rapidly translate into equipment damage, unplanned downtime, and — in the worst case — safety incidents.

The objective of this project was to architect a robust, fault-tolerant control system for an automated palletizer that actively anticipates and mitigates these physical anomalies, rather than merely reacting to them. A modular PLC programming approach isolates state management from physical I/O, ensuring mechanical actuators only operate under strictly verified conditions. Comprehensive HMI visualization and active alarm management were integrated on top of this logic layer to bridge the gap between raw machine execution and operator awareness — a gap that, in most legacy timer-based systems, is left entirely to guesswork on the plant floor.

2. System Overview

The palletizing cell is a fully integrated automation environment that cleanly separates three domains: the physical plant, the control logic, and the operator interface.

- **Physical Plant —** the electromechanical behavior of the system, including conveyor motors, pneumatic pushers, limit switches, and photoelectric sensors.
- **Logic Execution (S7-1200) —** the PLC continuously evaluates field sensor conditions against the core state machine, calculates the required mechanical transitions, and drives the actuator outputs.
- **Operator Interface (WinCC HMI) —** sits above the control architecture and monitors centralized global data blocks, translating raw PLC tags into real-time machine states, production metrics, and discrete active faults for direct operator interaction and secure manual intervention.

3. Automation Architecture

The control system is built on a distributed architecture that isolates logic solving, operator visualization, and field I/O into distinct functional layers:

- **Logic Controller —** a Siemens S7-1200 (CPU 1214C DC/DC/DC) serves as the core logic solver, processing field inputs, managing the internal state machine, and driving output signals within a deterministic scan cycle.
- **Engineering Environment —** system programming, hardware configuration, and HMI development were executed concurrently within TIA Portal V16, ensuring seamless tag routing between the PLC and visualization layers.
- **Visualization Node —** a WinCC Basic HMI operates as the primary supervisory interface, continuously reading centralized PLC Data Blocks to display sequence status, active modes, and diagnostic alarms without interfering with the deterministic control loop.
- **Field I/O —** the physical layer consists of discrete digital inputs (photoelectric sensors, limit switches, E-Stops) and discrete digital outputs (motor contactors, pneumatic solenoids).

4. PLC Software Architecture

To ensure scalability, maintainability, and code reusability, the PLC software abandons flat, centralized logic in favor of a highly modular, object-oriented approach. The architecture is structurally divided into hierarchical layers: global management, sequence control, and reusable equipment modules.

Figure 1 — Block and Data Structure

```
PLC_1 (CPU 1214C DC/DC/DC)
│
├── Organization Blocks (OB)
│   └── Main [OB1]                 // Cyclic execution & routing
│
├── Simulation Interface (FC)
│   └── MHJ-Lab-Function-S71200 [FC9000]  // I/O to sim. plant
│
├── System Management & Sequencing (FB)
│   ├── FB_ModeManager [FB4]         // Auto/Manual, E-Stop eval.
│   ├── FB1_Pallet_Handling [FB1]    // Pallet dispenser & exit
│   ├── FB2_Box_Grouping [FB2]       // Infeed & row forming
│   └── FB3_Elevator_Stacker [FB3]   // Layer drops & array math
│
├── Reusable Equipment Modules (FB)
│   ├── FB_Cylinder [FB5]            // Standard pneumatic logic
│   └── FB_Motor [FB6]               // Standard motor/drive logic
│
├── Global Data Blocks (DB)
│   └── DB_Machine [DB12]            // Centralized tags (HMI)
│
└── Instance Data Blocks (IDB)
    ├── Management — FB_ModeManager_DB [DB11]
    ├── Sequences  — FB1_Pallet_Handling_DB [DB1], FB2_Box_Grouping_DB [DB3]…
    ├── Motors     — FB_Motor_BoxFeed [DB16], InstDB_Motor_ElevUp [DB19]…
    └── Cylinders  — InstDB_Cylinder_Pusher [DB13], InstDB_Cyl_Clamp [DB14]…
```

Architectural Rationale and Engineering Decisions

- **Object-Oriented Equipment Control —** rather than writing unique rung logic for every physical actuator, standardized equipment Function Blocks (FB_Motor and FB_Cylinder) encapsulate all standard actuator behavior — manual jogging, interlocks, and diagnostic watchdogs — in one place. Each block is instantiated multiple times with a unique Instance Data Block (e.g., InstDB_Cylinder_Pusher vs. InstDB_Cyl_Clamp), drastically reducing code duplication and standardizing fault behavior across the machine.
- **Hierarchical Sequence Modularity —** the machine sequences are physically decoupled: FB1_Pallet_Handling operates entirely independently of FB2_Box_Grouping, allowing the infeed conveyor to keep accumulating boxes while the pallet dispenser is indexing — maximizing theoretical machine throughput. FB3_Elevator_Stacker bridges the two systems, taking the formed layers and dropping them onto the waiting pallets.
- **Simulation Interfacing —** FC9000 cleanly isolates the communication handshakes required between the S7-1200 logic and the virtual plant simulation, keeping the core machine logic completely independent of the simulation drivers.

5. Functional Block Roles and Logic Modules

The control architecture is divided into distinct operational zones and reusable equipment modules. Detailed ladder logic and SCL networks for each block are documented in the accompanying exported PDFs in the project repository.

System Management Layer

- **FB4_ModeManager —** the global safety and state gatekeeper. It evaluates the physical panel selector switches and E-Stop circuits to determine whether the system is in Auto, Manual, or a Faulted state, and broadcasts this status to every other block.

Sequence Control Layer

- **FB2_Box_Grouping —** manages the upper-level infeed sequence, forming and safely indexing box rows into the elevator zone using closed-loop sensor verification.
- **FB1_Pallet_Handling —** manages the ground-level pallet sequence. Operating independently of box grouping, it dispenses empty pallets and controls the heavy exit conveyor once a pallet is fully stacked.
- **FB3_Elevator_Stacker —** the bridging sequence. It synchronizes with FB1 and FB2 to calculate layer math, lower the elevator plate, and drop the formed box layers onto the waiting pallet.

Reusable Equipment Modules (Object-Oriented)

- **FB_Motor & FB_Cylinder —** standardized control blocks engineered for every physical actuator. Rather than unique logic per device, these blocks encapsulate standard auto-commands, manual jogging, thermal-fault logic, and hardware cross-interlocks, and are instantiated dynamically throughout the project (e.g., DB16 for Box Feed, DB13 for the Pusher Cylinder).

6. Deep Dive — FB3 Elevator Stacker: The Core SCL State Machine

While the infeed and pallet-handling sequences operate independently, FB3_Elevator_Stacker acts as the central master synchronizer. Written entirely in Structured Control Language (SCL), this block leverages advanced programmatic structures to manage array math, dynamic layer patterning, and closed-loop sequence execution. It is responsible for four critical automation tasks.

6.1 Dynamic Layer Patterning (Modulo Math)

To preserve the physical stability of the stacked pallet, boxes cannot be stacked in identical columns layer after layer. FB3 dynamically generates interlocking layer patterns using a modulo function (MOD 2) against the current layer count:

- Even layers (0, 2): a pattern of 3 boxes per row across 2 rows, with the pneumatic turner disabled.
- Odd layers (1, 3): the pattern shifts to 2 boxes per row across 3 rows, automatically enabling the pneumatic box turner (M_Enable_Turn := TRUE) to physically rotate the incoming product.

6.2 Deterministic State Management (CASE Execution)

To prevent mechanical race conditions — such as the elevator attempting to drop before the clamping plates are fully extended — the sequence is governed by a strict CASE statement.

- **Physical verification:** the state machine mathematically waits for mechanical confirmation. For example, in State 80 (Discharge Pallet), the sequence sets the command but will not advance to State 81 until the I_Elevator_Moving sensor physically verifies the drive has energized.
- **Single-scan math protection:** State 30 handles the row-incrementing logic. It executes the mathematical addition exactly once before immediately forcing a jump to State 31 (Wait for Clamp), eliminating the risk of repeated or runaway math within a single PLC scan.

6.3 Parallel Diagnostic Watchdogs & Slice-Access Alarms

Protecting the machine from physical jams is paramount. Instead of relying on the sequence states themselves to time out, FB3 runs parallel TON watchdogs outside the CASE statement:

- If the pneumatic drop plate fails to open within 4.0 seconds (State 60), the #stat_Plate_Timeout timer triggers a specific #stat_Fault_ID := 3.
- The logic then uses Siemens slice-access addressing — #io_stState.wAlarmWord.%X3 := (#stat_Fault_ID = 3) — to map that integer fault code directly onto a discrete bit of the global Alarm Word, instantaneously triggering the correct diagnostic text on the WinCC HMI.

6.4 Hard Software Interlocks

At the very top of the block's execution, the logic evaluates the global xRunning permissive. If the Mode Manager detects an E-Stop or fault condition, FB3 immediately forces all internal auto-commands to FALSE, zeroes the active counts, and executes a RETURN instruction. This halts the block's scan instantly — a hard software interlock that prevents the CASE statement from executing at all.

Code Snippet — Dynamic Patterning and Watchdog Execution (FB3)

```
// --- Watchdog Timers (Parallel to State Machine) ---
```

```
#EjectTimeout(IN := #stat_Step = 81, PT := T#8.0S);
IF #EjectTimeout.Q THEN
    #stat_Fault_ID       := 4;
    #io_stState.xFaulted := TRUE;
    #stat_Step           := 0;              // Abort Sequence
END_IF;

// --- Alarm Word Mapping (Slice Access) ---
#io_stState.wAlarmWord.%X4 := (#stat_Fault_ID = 4);

// --- Dynamic Pattern Generator ---
IF (#stat_Layer_Count MOD 2) = 0 THEN
    "M_Target_Boxes" := 2;
    "M_Target_Rows"  := 3;
    "M_Enable_Turn"  := FALSE;              // Even Layer Pattern
ELSE
    "M_Target_Boxes" := 3;
    "M_Target_Rows"  := 2;
    "M_Enable_Turn"  := TRUE;               // Odd Layer Pattern (Interlocked)
END_IF;
```

7. WinCC HMI and Operator Interface (SCADA)

A robust control architecture requires an equally robust visualization layer. The WinCC Basic HMI was engineered to prioritize operator situational awareness, diagnostic speed, and secure fault recovery — moving far beyond basic start/stop functionality.

To preserve the deterministic scan time of the S7-1200, the HMI does not poll individual I/O points scattered throughout the program. Instead, it exclusively reads from and writes to the centralized DB_Machine global data block.

Key Interface Features

- **Real-time process dashboard —** the main screen provides live feedback of the active process, reading the integer state of the machine (iCurrentStep) and the calculated data from FB3 (such as io_stState.iCurrentLayer and total box counts), translated into dynamic visual indicators and production metrics.
- **Discrete alarm management —** leveraging the slice-access mapping engineered in the core logic (e.g., io_stState.wAlarmWord.%X4), the HMI translates raw PLC bits into specific, timestamped diagnostic messages (e.g., “Pallet Discharge Jam”). Operators acknowledge faults via a dedicated xHmiAckFault software button, logically OR'd with the physical panel reset.
- **Secured manual override —** because manual operation of pneumatic clamps and heavy conveyors presents a safety risk, the Manual Control screen is restricted via Siemens User Administration and requires a valid engineering or maintenance password. Once authenticated, operators can jog individual actuators — but only if FB_ModeManager confirms the physical selector switch is natively set to Manual mode.

8. Validation Methodology and Results

Deploying unverified code directly to a physical palletizer often results in mechanical collisions, damaged product, or compromised safety. To validate the closed-loop SCL architecture and the parallel watchdog interlocks, the PLC logic was interfaced with a high-fidelity electromechanical simulation in Factory I/O — effectively a digital twin of the mechanical cell.

System Validation

The digital-twin environment allowed for aggressive stress testing of the control system against real-world physical variables:

- **Sensor delays & coasting —** confirmed that the CASE sequence correctly halted when photoelectric sensors experienced simulated dirty lenses or misalignment, rather than allowing the machine to proceed on a false assumption of position.
- **Mechanical jams —** box jams were intentionally introduced on the infeed conveyor, and the pallet discharge was deliberately blocked. In both cases the parallel TON watchdogs successfully intercepted the timeouts, halted the actuators, and generated the correct slice-access alarm codes without locking up the main sequence math.

9. Conclusion

This project demonstrates the design and commissioning of an industrial-grade automated palletizing cell. By moving away from open-loop ladder timers and embracing object-oriented PLC programming, the resulting Siemens S7-1200 architecture is highly modular, scalable, and inherently fault-tolerant. The clean separation of physical I/O mapping, centralized data structures, and deterministic SCL state management shows a control system engineered not just to run the process, but to actively protect its own mechanical hardware and minimize downtime.

> **Repository & Documentation**
> Full ladder logic exports, SCL source (FB3_Elevator_Stacker.scl), and function block PDFs (FB_Motor, FB_Cylinder, FB1–FB4) are available in the project repository for detailed review.